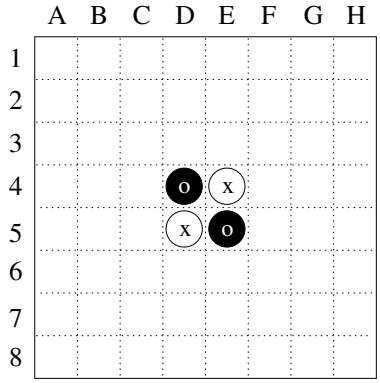
Pràctica 2: Algoritmos Minimax y $\alpha - \beta$. Heurísticas para juegos.

1. Plantejament del problema

El primer objectiu d'aquesta pràctica, per a desenvolupar en dues sessions, és entendre i implementar l'algorisme $\alpha - \beta$ per a resolució en arbres de joc. L'algorisme haurà d'implementar-se del mode més general possible, però es provarà aplicant-lo al joc de l'Otello, per al qual a més d'han de realitzar diverses heurístiques de joc i fer-les competir les unes contra les altres, tant les realitzades per components de la mateixa parella de pràctiques com les d'altres parelles.

Les regles de l'Otello són: siga un tauler quadrat de 64 caselles (8 files i 8 columnes). Existeixen 64 fitxes en forma de moneda amb colors o símbols diferents en cada cara (com les monedes reals). En el nostre cas, diguem una cara blanca i una negra. La configuració inicial és la que es mostra en la figura.



A cada jugador se l'associa un color (cara) de les fitxes. Començant pel jugador a qui s'assigna negre, cada jugador situa una fitxa en qualsevol de les caselles lliures amb la condició que en fer-ho es puga revertir almenys una fitxa del color de l'adversari. Una fitxa s'ha de revertir si està alineada en horitzontal, vertical o diagonal entre dues del color oposat, i sempre que tota la línia entre aquestes siga contínua (sense buits) i del mateix color. Això significa p. ex. que des de la posició d'inici, la primera jugada de negra només podria ser a E3 (voltejant llavors la blanca en E4), a C5 (voltejant D5), a F4 (voltejant E4) o a D6 (voltejant D5).

Si un jugador no pot col·locar cap fitxa del seu color d'acord amb aquestes regles, ha de passar, i només en aqueix cas se li permet fer-lo. El joc acaba quan les 64 fitxes estan col·locades, o quan cap dels dos jugadors pot col·locar cap fitxa més. Gana el jugador el color del qual es mostre en aqueix moment en més fitxes del tauler, o els dos jugadors empaten en cas d'igualtat en nombre de fitxes.

El resultat lliurable és:

- La implementació de l'algorisme $\alpha - \beta$ i la seua prova en el joc.
- La implementació d'almenys dues heurístiques i la seua prova en el joc jugant una contra l'altra.
- Un arxiu `explic.txt` en text ASCII amb les instruccions de compilació i resultats obtinguts amb cada heurística.

S'entregarà el codi realitzat PROU COMENTAT.

Apèndix: API de C++

Es defineixen alguns tipus i es proporciona una classe `Board` per a facilitar la implementació.
Els tipus són:

- Tipus Square: `typedef pair<char,char>Square`. És un parell de dos caràcters que representa una casella del tauler. Per a ser vàlid el primer caràcter ha de ser '1',...,'8' (la fila), i el segon, 'A',...,'H' (la columna). Note's que els números estan representats com a caràcters, no com a enters.
- Tipus Sqlist: `typedef list<Square>Sqlist`. Una llista de caselles.
- Tipus Token: definit per enumeració com `enum Token {x,o,e }`. L'o representa negre, la x blanc i l'e buit (empty).

- Tipus Player: definit per enumeració com `enum Player {Bob,Alice}`. Per conveni, Alice juga amb negres (token 'o') i per tant obri el joc.
- Tipus Move: `typedef pair<Player,Square>Move`. Representa un moviment, on la primera component és el jugador que el realitzarà i la segona, la casella on posarà la fitxa del seu color.

Les funcions auxiliars són:

- `string Name(Player pl)`: Presa un jugador (com a tipus Player) i retorna una string amb el seu nom. (Pensada per a mostrar per pantalla).
- `Player Opponent(Player pl)`: Presa un jugador com a tipus Player i retorna el seu oponent, també com a tipus Player.
- `Token Tk(Player pl)`: Presa un jugador com a tipus Player i retorna el valor (com a tipus Token) de la fitxa amb les quals juga.
- `char Symbol(Token tk)`: Presa una fitxa (com a tipus Token) i retorna un caràcter. (Pensada per a imprimir per pantalla).
- `bool ValidSquare(Square sq)`: Presa una casella i retorna true si és vàlida i false si no. Vegeu a dalt les restriccions als caràcters del parell per al tipus Square.
- `bool ValidPlayer(Player pl)`: Presa un jugador i retorna true si és un dels dos jugadors vàlids i false si no.

La classe es diu `Board`, i el seu interface públic és:

- `Board()`: Crea un tauler en l'estat inicial (és a dir, amb dues fitxes 'o' en ('4','D') i ('5','E') i dues fitxes 'x' en ('5','D') i (4,'E')).
- `void Show(ostream &outst)`: Mostra per pantalla (passant cout o cerr) o en qualsevol altra stream d'eixida l'estat actual del tauler.
- `Token Content(Square sq)`: Retorna el token (x,o,e) que ocupa la casella que se li passa.
- `bool IsLegal(Move mv)`: Retorna `true` si el moviment passat pot executar-se en l'estat actual del tauler i `false` si no.
- `void MakeMove(Move mv)`: Efectua el moviment que se li passa, si és legal (no és necessari comprovar per endavant, aquesta funció ho fa), la qual cosa altera l'estat intern del tauler (de l'objecte que crida a aquesta funció).
- `Board TryMove(Move mv,bool &legal)`: Si el moviment que se li passa és legal (el comprova internament), còpia el tauler actual en un nou, executa el moviment sobre aquest, i retorna aquesta còpia, amb el que l'objecte que crida a aquesta funció no s'altera. La variable `legal`, passada per referència, torna amb el valor obvi segons el moviment puga executar-se o no.
- `bool GameOver(Player &winner,unsigned &winnerpoints,unsigned &looserpoints)`: Retorna `true` si el joc ha acabat, és a dir, si cap dels contendents pot realitzar més jugades (per bloqueig, o perquè el tauler està ple) i `false` en cas contrari. A més, si el joc ha acabat, les variables passades per referència tornen plenes amb respectivament el guanyador (com a tipus Player), el nombre de fitxes del guanyador i el número del perdedor (com a tipus `unsigned`).
- `Sqlist ValidMoves(Player pl)`: retorna una llista de caselles en les quals al jugador que se li passa se li permet posar una fitxa (per descomptat, del seu color)

La funció heurística està definida en els arxius `heuristic.h/cpp`. Es proveeix l'heurística no informativa (retorna sempre 0) i l'exercici és fer una altra o altres millors, usant les funcions que es considere necessari de la classe `Board` (essencialment, `Content` per a veure quines fitxes hi ha en el tauler i on estan). Haurien de programar-se almenys dues heurístiques, i fer-les jugar l'una contra l'altra. Per a això la funció heurística pot implementar-se en la forma

```
#include "heuristic.h"

float h(Player pl,Board b)
{
 if (pl==Bob)
 {
 // Program one heuristic here...
 // return ...
 }
 else
 {
 // Program a different one here...
 // return ...
 }
}
```

A més, el programa principal juga amb un algorisme minimax, definit en `Minimax.h/cpp`. La implementació està detalladament comentada. L'exercici és copiar aquests dos arxius en uns altres, `Alfabeta.h/cpp` i modificar tot el necessari per a escriure un $\alpha - \beta$, provant-ho després en el programa principal en substitució del minimax.